SYNAPSEOS · THESIS 1

Revisions After the Proposal Defense

12 October 2026, updated after Dr. Rey's review · Alexandra Sulit, Willard Soriano, Steven Evian Lozano · Page numbers refer to the attached revised proposal (122 pages).

Response to the Panel

CONCERN	RESPONSE	WHERE
Chapter 1 has no Background of the Study.	The content was present but had no subsection headings. Chapter 1 is now divided into numbered subsections, beginning with 1.1 Background of the Study, as Chapters 2 and 3 are.	Ch. 1, pp. 1–14
How is this a computer science study rather than an IT project?	The core contribution is an algorithm: a static analysis that predicts what a command will change before it runs, so that what it destroys can be backed up and restored. It builds on established program-analysis theory and is evaluated independently of the interface.	pp. 24, 62, 74
The objectives are not aligned with Computer Science; objectives 4–5 were system design and HCI. (Sir Joel; Dr. Rey)	Adopted Dr. Rey's five objectives, all algorithmic: an effect-semantics model, an effect-composition algorithm, a minimum-cost recovery plan, evaluation against pattern-based and fail-closed baselines (on data loss, verified restoration, and needless confirmation), and ablation. Each has its own research question. SynapseOS is now the experimental platform, and the user study validates the algorithm in use. New title: *Recoverability Analysis of Language-Model-Generated Shell Commands, Deployed in the SynapseOS Conversational Interface*.	§1.3–1.4, pp. 5–8; §3.4(e), p. 90
How much of the result comes from the algorithm, and how much from its rule tables? (adviser, earlier review)	Now measured for both of the algorithm's parts. With the same rule table, backups fall from **89.5%** to **35.4%** without composition, **45.5%** without runtime-target resolution, and **32.0%** with neither (each $p < 10^{-42}$). Data loss without warning stays 0% in every case. The algorithm, not the table, drives the result.	§3.1(h), pp. 84–87

New Evidence at a Glance

- **The algorithm drives the result:** backups drop from 89.5% to 32.0% without composition and resolution, and each part needs the other (pp. 84–87).
- **Our baseline is not a weak one:** the safety checks in two open-source AI coding tools, built for code projects where git can undo mistakes rather than for personal files, let 73–97% of file-destroying commands through without asking, against 31% for our baseline and 0% for the algorithm (p. 78).
- **It works end to end:** run through the full system on real files, undo restored deleted files only with the algorithm enabled (p. 70).

Limitations We Disclose

- Undo currently reverts the most recent change regardless of folder; it will be scoped before data collection (p. 93).
- The small local language model sometimes mistranslates loosely worded requests, as already noted in the scope (p. 14); this does not affect the algorithm.

Supporting material (the comparison with open-source tools, a recorded end-to-end verification, and per-command results for all 777 test commands) is available on request.